CS315 · ALGORITHMS ANALYSIS & DESIGN

Practice 1 Analysis & Sorting الحل

تدريب على Lecture 1 و Lecture 2 · 11 سؤال

محارب

01 حل Q1: arraySum

السطر	# operations	ليش
`sum ← 0`	1	assignment.
`for i ← 0 to n − 1 do`	1 + (n + 1)	تهيئة مرة، والمقارنة `n + 1` مرة لأن اللوب يلف `n` مرة.
`sum ← sum + A[i]`	3n	indexing + جمع + assignment.
`{ increment counter i }`	2n	جمع + assignment.
`return sum`	1	return.
Total	**6n + 4**	`O(n)`

انتبه

هنا اللوب يبدأ من 0 فيلف `n` مرة. في **arrayMax** يبدأ من 1 فيلف `n − 1` مرة. دايماً احسب عدد اللفات أول.

02 حل Q2: arrayMax مع Temp

Best	Worst	السطر
2	2	currentMax ← A[0]
2	2	Temp ← A[0]
1 + n	1 + n	for i ← 1 to n − 1 do
2(n − 1)	2(n − 1)	if A[i] > currentMax then
0	2(n − 1)	currentMax ← A[i]
2(n − 1)	2(n − 1)	{ increment counter i }
1	1	return currentMax
5n + 2	**7n**	**Total**

```
Worst:   2 + 2 + (1 + n) + 6(n − 1) + 1  =  7n
```

```
Best:    2 + 2 + (1 + n) + 4(n − 1) + 1  =  5n + 2
```

هذا نفس سؤال ميد 451 و 442. الجواب المطلوب هناك `7n`، يعني جواب المحاضرة `7n − 2` زائد عمليتين من سطر Temp.

03 حل Q3: Nested loops

السطر	# operations	ليش
`count ← 0`	`1`	
`for i ← 0 to n − 1 do`	`n + 2`	`1 + (n + 1)`.
`for j ← 0 to n − 1 do`	`n(n + 2)`	السطر كله يتنفذ `n` مرة، وكل مرة يكلف `n + 2`.
`count ← count + 1`	$2n^2$	عمليتين، `n × n` مرة.
`{ increment counter j }`	$2n^2$	
`{ increment counter i }`	$2n$	
`return count`	`1`	
Total	$\mathbf{5n^2 + 5n + 4}$	$O(n^2)$

$$1 + (n + 2) + (n^2 + 2n) + 2n^2 + 2n^2 + 2n + 1 \;=\; 5n^2 + 5n + 4$$

04 حل Q4: تعريف Big-Oh

a $O(n)$ is $5n + 3$

$$5n + 3 \leq cn$$

$$(c - 5)n \geq 3$$

Pick $c = 6$: $n \geq 3$ → $c = 6$, $n_0 = 3$

b $O(n^2)$ is $4n^2 + 3n + 2$

$$4n^2 + 3n + 2 \leq 4n^2 + 3n^2 + 2n^2 = 9n^2 \quad \text{for } n \geq 1$$

$c = 9$, $n_0 = 1$

c $O(\log n)$ is $3 \log n + 5$

For $n \geq 2$: $\log n \geq 1$ so $5 \leq 5 \log n$

$$3 \log n + 5 \leq 3 \log n + 5 \log n = 8 \log n$$

$c = 8$, $n_0 = 2$

d — $O(n^2)$ is not n^3

$$n^3 \leq cn^2$$

$$n \leq c$$

مستحيل تتحقق لكل $n \geq n_0$ ، لأن c ثابت و n تكبر بلا حد.

أي قيم ثانية تحقق المتراجحة تعتبر صحيحة. مثلاً في (a) يصح $c = 8, n_0 = 1$.

05 حل Q5 و Q6

$$Q5: \quad 100 < \log n < 5n < n \log n < 4n^2 < n^3 < 2^n$$

#	f(n)	Big-Oh	السبب
a	$10^5n^2 + 10^8n$	$O(n^2)$	نحذف الثوابت والحد الأصغر.
b	$n(n + 1) / 2$	$O(n^2)$	تساوي $n^2/2 + n/2$.
c	$3 \log n + \log \log n$	$O(\log n)$	$\log \log n$ أصغر من $\log n$.
d	$7n - 2$	$O(n)$	polynomial درجته 1.
e	$2^n + n^3$	$O(2^n)$	الأسّي أسرع من أي polynomial.
f	500	$O(1)$	ثابت.

06 حل Q7: True / False

#	الجواب	السبب
1	**F**	لازم نفس الـ hardware والـ software **ونفس المدخلات**.
2	**T**	نص السلايد.
3	**T**	نعدّ أكبر عدد عمليات ممكن، وهذا الـ worst case.
4	**F**	يغيّر `T(n)` بـ constant factor فقط.
5	**F**	`n² ≤ cn` تعطي `n ≤ c`، مستحيل.
6	**F**	في كل الحالات.
7	**T**	لما الأرقام مرتبة أصلاً.
8	**F**	هذا وصف Merge sort.
9	**T**	تعريف الـ RAM model.
10	**F**	Ω معناها أكبر من أو تساوي. الأقل من أو تساوي هي O.

الجمل 1 و 2 و 3 جت بالنص في الميدات. ما عندي نموذج إجابة رسمي، والأجوبة من السلايدات.

07 حل Q8: Prefix averages

```
a)   A[0] = 4 / 1 = 4
     A[1] = (4 + 8) / 2 = 6
     A[2] = (4 + 8 + 6) / 3 = 6
     A[3] = (4 + 8 + 6 + 2) / 4 = 5          A = [4, 6, 6, 5]
```

b) لأنها تعيد حساب المجموع من البداية لكل `i` . اللوب الداخلي يلف `1 + 2 + … + (n − 1)` مرة، وهذا مجموع arithmetic progression يساوي `n(n − 1)/2` ، فالوقت `O(n²)` .

```
c)  Algorithm prefixAverages2(X, n)
        A ← new array of n integers
        s ← 0
        for i ← 0 to n − 1 do
            s ← s + X[i]
            A[i] ← s / (i + 1)
        return A
```

08 حل Q9: Insertion sort

i	key	Array after the iteration
–	–	5 2 4 6 1 3
2	2	2 5 4 6 1 3
3	4	2 4 5 6 1 3
4	6	2 4 5 6 1 3
5	1	1 2 4 5 6 3
6	3	1 2 3 4 5 6

b) الـ best case لما المصفوفة مرتبة أصلاً: `O(n)` . الـ worst case لما تكون مرتبة بالعكس: `O(n²)` .

c) **Initialization** ثم **Maintenance** ثم **Termination**.

09 حل Q10: Merge sort

Divide	[7 2 9 4 3 8 6 1]	
	[7 2 9 4]	[3 8 6 1]
	[7 2] [9 4]	[3 8] [6 1]
	[7] [2] [9] [4]	[3] [8] [6] [1]
Merge	[2 7] [4 9]	[3 8] [1 6]
	[2 4 7 9]	[1 3 6 8]
	[1 2 3 4 6 7 8 9]	

```
b)   T(n) = T(n/2) + T(n/2) + T(MERGE)
```

```
c)   Levels      =  log(16) + 1  =  4 + 1  =  5        (levels 0 to 4)
```

```
     Level 3     =  2³ = 8 problems,  each of size 16 / 2³ = 2
```

```
     Total work  =  6n(log(n) + 1)  =  6 × 16 × 5  =  480
```

10 حل Q11: Divide and conquer

a) **Divide**: نقسم المشكلة إلى sub-problems من نفس الشكل. **Conquer**: نحلها recursively.

Combine: نجمع الحلول.

	Merge sort	Quick sort
التقسيم	من النص، نصين متساويين.	حول pivot، وممكن غير متساوي.
مصفوفة إضافية	يحتاج.	ما يحتاج.
خطوة الدمج	يحتاج.	ما يحتاج.
Worst case	$O(n \log n)$	$O(n^2)$